

ОСНОВЫ РАБОТЫ С СУБД ДЛЯ СПЕЦИАЛИСТА ПО ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

Введение в раздел

Современные информационные системы для управления данными используют, как правило, специальные программные средства – СУБД, системы управления базами данных. Это верно не только для массовых, так называемых «промышленных» систем или для их облегченных копий, например, систем веб-ресурсов. Даже в решениях высокой нагрузки используются СУБД, просто специализированные под тот или иной вид задач, или их ансамбли.

Не смотря на все многообразие систем управления базами данных, общие принципы и подходы к их построению остаются неизменными. В основе каждой такой системы лежит некоторая модель данных и набор инструментов по манипулированию информацией, структурированной в соответствии с этой моделью.

При построении защищенных баз и банков данных важно не только включать и настраивать наложенные средства защиты или встроенные возможности, например, по аутентификации и авторизации пользователей. Так как системы управления данными создавались, в первую очередь, для решения производственных задач, средства безопасности в них зачастую оставляют желать лучшего, за редким исключением специализированных версий. Однако, на основе встроженных механизмов развитых современных СУБД можно выстраивать достаточно эффективную эшелонированную защиту от большого числа злоумышленников, не обладающих глубокими специализированными знаниями. В то же время, использование средств защиты всегда приводит к снижению производительности СУБД, как «платы» за безопасность. И задача специалиста в этом случае – найти баланс защищенности и производительности в соответствии с конкретной задачей и требованиями.

Таким образом, эффективное решение задач безопасности невозможно без понимания принципов функционирования систем управления данными и умения использовать их основные компоненты. Именно этим вопросам посвящен первый раздел практикума. Тем не менее, помимо навыков работы с СУБД, он также включает и практические задания по базовым методам обеспечения безопасности: разграничению доступа и защите от одной из самых популярных (уже много лет!) атак – инъекции в язык запросов или SQL – инъекции (SQLi).

Работы этого раздела являются базовыми и могут использоваться как самостоятельный лабораторный курс при подготовке бакалавров или

специалистов со средним профессиональным образованием в области информационной безопасности.

Первые три работы раздела развивают навыки моделирования данных, работы с хранимой информацией и разграничения доступа. На практике для студентов, знакомящихся с базами данных и СУБД впервые, порядок работ 2 и 3 рекомендуется изменить. И, после работы по организации данных, провести работу по манипулированию ими, а только затем знакомиться с продвинутыми инструментами. Для специалистов знакомых с СУБД и развивающих именно навыки защиты баз данных рекомендуется следовать порядку работ, приведенному в практикуме.

Работы 4 и 5 посвящены задачам обеспечения производительности СУБД под нагрузкой и, в зависимости от специфики курса подготовки, могут быть опущены или сделаны рекомендованными (факультативными). Их рекомендуется выполнять в порядке, приведенном в практикуме, так как понимание физического хранения и работы индексов существенно повышает осознанность и эффективность работы с планами запросов.

Работа № 1. Организация данных и разграничение доступа

Общие сведения о работе

Цель работы - получение навыков создания схемы данных и базового разграничения доступа при работе с СУБД.

Моделирование данных в СУБД

В программах для хранения и обработки данных, таких как системы управления базами данных, используются различные способы формализации сведений реального мира, называемые моделями данных. В строгом понимании модель данных это совокупность методов и средств определения логической структуры базы данных и динамического моделирования состояния предметной области в базе данных. В модели данных как правило выделяют три компонента:

- 1) средства определения допустимых структур данных;
- 2) множество операций, применимых для поиска и модификации данных, к базе данных, находящейся в допустимом состоянии;
- 3) множество ограничений целостности, определяющих множество допустимых состояний базы данных.

Также существует более формализованное понятие математической модели данных, определяющей общие принципы оперирования информацией системы управления базами данных. Основа математических моделей была заложена Коддом при создании реляционной алгебры и реляционного исчисления.

При практической работе с базами данных выделяют несколько уровней описания (представления) данных. Эти уровни соответствуют общей архитектуре СУБД и представляют собой три слоя: концептуальный (пользовательский), логический и физический (рисунок 1.1).



Рисунок 1.1. Архитектура СУБД ANSI/SPARC

Концептуальный уровень представляет данные в виде понятий пользовательской сферы и может быть по-разному детализирован. При проектировании баз данных концептуальный пользовательский уровень представляется в виде ER-схемы, или диаграммы сущность – связь.

Логический уровень представляет собой структуры данных и работу с ними на уровне манипулирования СУБД. В данном практикуме в качестве логической рассматривается реляционная модель.

Физический уровень описывает манипулирование данными на уровне файлов и файловых записей, структур в оперативной памяти. Частично этот вопрос затрагивается в работе по исследованию индексов в реляционной СУБД.

Реализация схемы данных на SQL

В реляционном сервере схема базы данных реализуется в виде скриптов на языке SQL. При этом лучше не пользоваться визуальными средствами проектирования с автоматической генерацией кода, так как в этом случае будет сложнее задать все необходимые характеристики схемы данных и обеспечить ее соответствие концептуальной модели.

Пример схемы в PostgreSQL (оболочка PGAdmin) приведен на рисунке 1.2. На рисунке представлено два отношения (prescription и pharmacy) относящиеся к схеме public. Указаны первичные ключи, поля отношений с типами данных и ссылка по внешнему ключу.

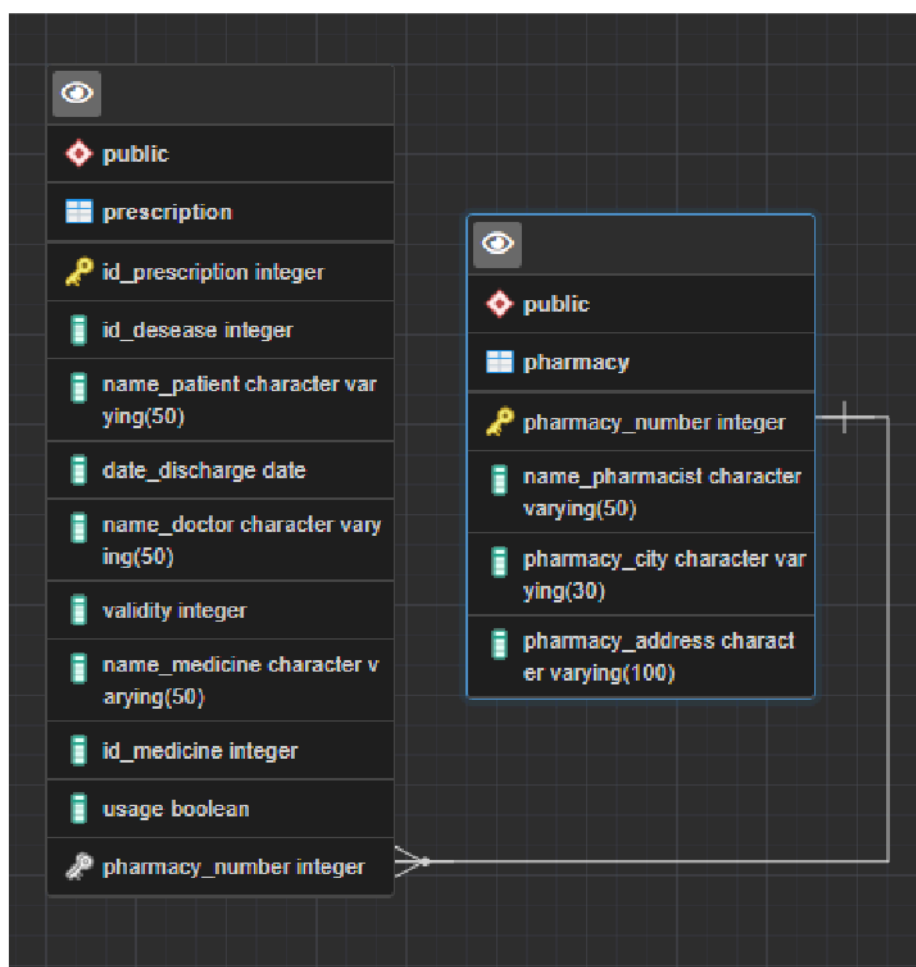


Рисунок 1.2. Схема данных в оболочке PGAdmin для PostgreSQL

В общем случае в РСУБД как правило атрибуты (поля) таблицы неявно упорядочены, тогда как записи хранятся без заданного порядка в зависимости от принятого физического расположения на диске (а не в порядке следования ключа). Число записей ограничено характеристиками аппаратно–программного обеспечения.

При задании отношения (таблицы) определяются его поля (столбцы), фактически, поля каждой записи. Для каждого поля указывается его тип, уникальность (если необходимо), возможность отсутствия значения и значение по умолчанию. Также при помощи CHECK приводятся ограничения уровня поля. Как правило, ограничения этого типа не могут содержать вложенных подзапросов.

Первичный ключ указывается при помощи специальной конструкции primary key. Для альтернативных - указывается их уникальность (unique) и невозможность отсутствия значения.

Единица языка SQL - оператор (statement), состоящий из нескольких предложений (clause). Для создания, изменения и удаления таблиц используется

часть SQL, называемая - язык определения данных (DDL, Data Definition Language).

Пользовательские базы данных как правило содержат два типа объектов:

1. Пользовательские объекты (таблицы, пользовательские типы данных, триггеры, хранимые процедуры, ограничения, индексы);
2. Системные объекты (таблицы, представления, хранимые процедуры, функции, типы данных, пользователи, роли, схемы).

Создать новую базу данных также можно при помощи SQL – скрипта. Рассмотрим пример создания базы данных на основе логической схемы, приведенной ниже на рисунке 1.3.

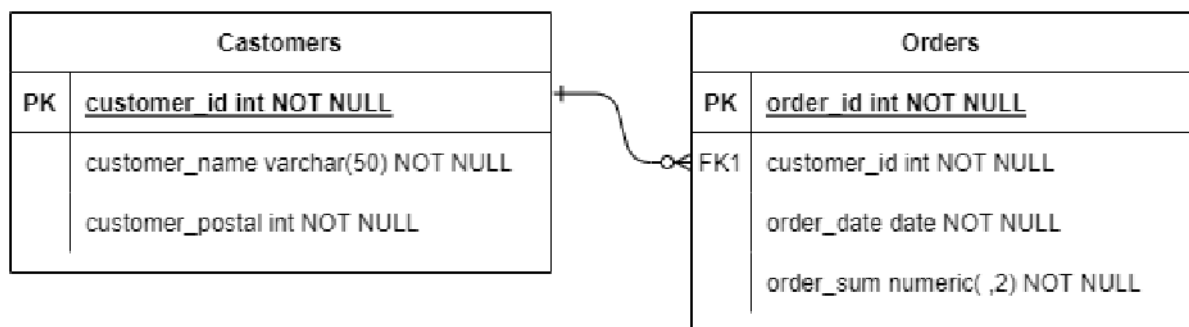


Рисунок 1.3. Пример схемы данных

Отношение Customers (Покупатели)

- customer_id - целое число, уникальный номер покупателя при регистрации, первичный ключ
- customer_name – фамилия покупателя, 50 символов
- customer_postal – почтовый индекс покупателя

Отношение Orders (Заказы)

- order_id – номер заказа, первичный ключ
- customer_id – номер покупателя, сделавшего заказ (из таблицы Customers)
- order_date – дата заказа в диапазоне от 01.01.2000 до 01.01.2022
- order_sum – сумма заказа, 2 знака после запятой, случайное число от 1 до 20 000

Код создания приведенных отношений:

```

DROP TABLE IF EXISTS Customers CASCADE;

CREATE TABLE Customers
(
    customer_id int primary key not null,
    customer_name varchar(50) not null,
    customer_postal int not null);

DROP TABLE IF EXISTS Orders CASCADE;

CREATE TABLE Orders
(
    order_id int primary key not null,
    customer_id int not null,
    order_date timestamp not null,
    order_sum numeric(7, 2) not null,
    foreign key (customer_id) references
Customers(customer_id)
    on delete cascade on update cascade);

```

При реализации схемы данных важен порядок создания таблиц. В частности, когда создается ссылка на таблицу (например, ссылка на таблицу Customers в таблице Orders) таблица, на которую ссылаются, должна уже существовать. Если необходимо создать циклические связи, то можно воспользоваться командой изменения таблицы с последующим добавлением ограничения (alter table... add constraint).

Конструкция DROP TABLE IF EXISTS позволяет сделать код создания таблиц пригодным к многократному запуску при внесении изменений и упрощает разработку.

Для создания суррогатных ключей и генерации абстрактных значений в СУБД используются генераторы последовательностей. Это объекты уровня базы данных синтаксис и специфика работы которых зависят от конкретной СУБД.

Вставка, удаление и обновление данных производится специальными SQL – операторами. Стандарт языка включает три основные операции добавления, удаления и изменения данных. Это INSERT, DELETE и UPDATE соответственно. Использование инструкции INSERT приведено в примере ниже.

```

INSERT INTO Customers values (1, 'Иванов', 170043);

```

Также для этого можно использовать функции языка, например, для заполнения таблиц случайными данными.

```

create or replace function insert_into_orders
    (num integer, start_value integer) returns boolean
as
$$begin
    if ($1 < 0 or $2 < 0) then return false;
        end if;
        for i in 1..$1 loop
            INSERT INTO Orders values
                ($2 + i, (select random_between(1,10)),
                (select timestamp '2000-01-01' + random() *
                (timestamp '2022-01-01' - timestamp '2000-
01-01'))),
                (select CAST(random_between_numeric(1,20000)
                as numeric(7,2))) );
        end loop;
        return true;
end$$
language plpgsql;

```

Ограничениями в реляционной модели являются так называемые ограничения целостности. В их основе лежат бизнес – правила предметной области и ссылочные ограничения СУБД, основанные на связях между отношениями и рассмотренные в предыдущей лабораторной работе по построению базы данных.

В общем случае *Ограничение целостности* – это логическое выражение, связанное с базой данных и принимающее значение TRUE (истина). Для семантической целостности оно выражает некоторое бизнес – правило.

В общем случае ограничения целостности могут быть наложены на тип данных (в том числе пользовательский тип), атрибут, отношение или несколько отношений (таблиц базы данных) – рисунок 1.4. С точки зрения особенностей реализации, ограничения целостности могут быть декларативными и не декларативными.

Декларативные ограничения целостности могут задаваться при создании или изменении таблиц базы данных или специальными операторами, определенными в языке SQL. К инструментам задания декларативных ограничений относятся:

Ключи отношений базы данных (первичные ключи; внешние ключи; атрибуты с уникальными значениями, представляющие альтернативные ключи отношений). Проверочные ограничения уровня атрибутов и отношений задаются оператором CHECK.

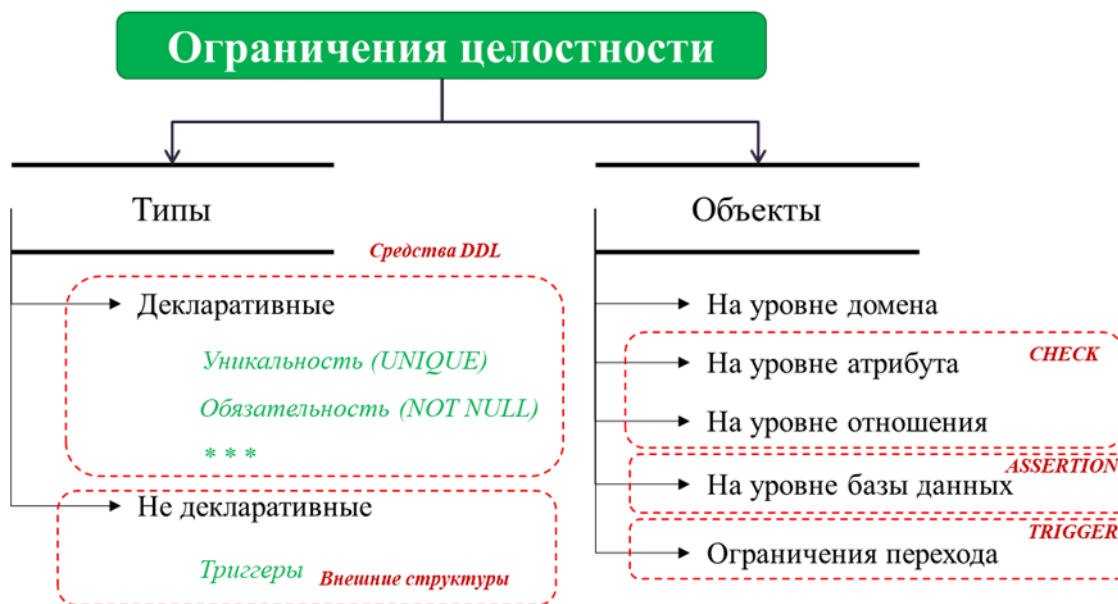


Рисунок 1.4. Виды ограничений целостности

Пример декларативного ограничения – обязательность поля:

```
CREATE TABLE products (
  product_no integer NOT NULL,
  name text NOT NULL,
  price numeric
);
```

Пример декларативного ограничения – уникальность поля:

```
CREATE TABLE products (
  product_no integer,
  name text,
  price numeric,
  UNIQUE (product_no, price)
);
```

Пример декларативного ограничения – CHECK на одно поле:

```
CREATE TABLE products
( product_no integer,
  name text,
  price numeric CHECK (price > 0)
);
```

Пример декларативного ограничения – именованный CHECK на одно поле:

```
CREATE TABLE products (
  product_no integer,
  name text,
  price numeric CONSTRAINT positive_price CHECK (price > 0)
);
```

Пример декларативного ограничения – CHECK на несколько полей:

```
CREATE TABLE products (
  product_no integer,
  name text,
  price numeric CHECK (price > 0),
  discounted_price numeric,
  CHECK (discounted_price > 0 AND price > discounted_price)
);
```


Ограничение CHECK не поддерживает вложенных запросов и не может, таким образом, охватывать несколько отношений. Ограничения уровня базы данных, которые нельзя выразить этим оператором, называются *недекларативными* и задаются при помощи триггеров.

Базовые средства разграничения прав доступа

В современных СУБД разграничение доступа к базе данных, или грануляция объектов, возможно на нескольких уровнях: уровне таблиц (TLS – table level security), уровне записей (RLS row level security) или уровне ячеек (CLS – cell level security) – рисунок 1.5. Большинство СУБД по умолчанию имеют только табличное разграничение доступа, но включают функциональные возможности для организации разграничения на уровне записей и ячеек.

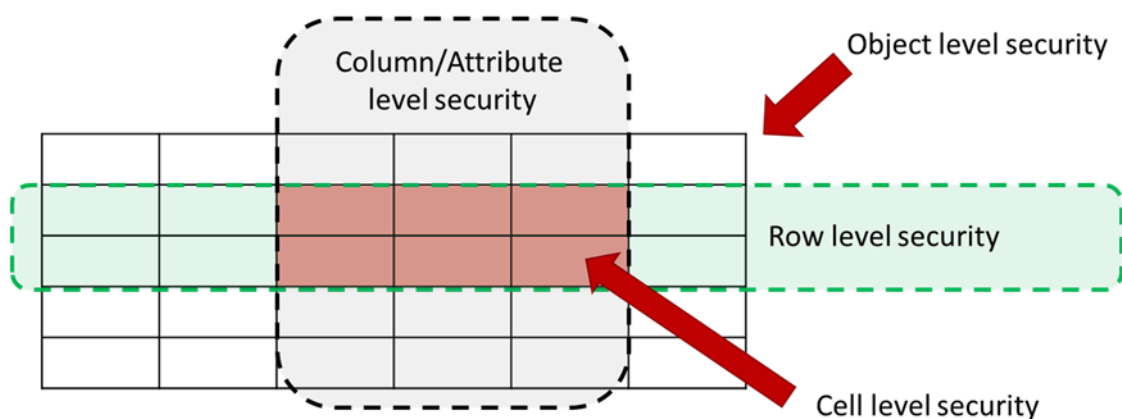


Рисунок 1.5. Виды разграничения доступа

В СУБД использующих не реляционную модель для логического представления данных грануляция доступа также связана с объектами, которыми манипулирует СУБД. В СУБД ключ – значение единицей управления является запись соответствующего типа и права доступа назначаются к записям на основе диапазона ключей. В СУБД класса семейство столбцов единицей доступа может служить запись или семейство записей (столбцов), также идентифицируемое ключом соответствующего уровня или именем.

При этом использование стандартного оператора и встроенных элементов зачастую является недостаточным как в плане детализации доступа (на уровне таблиц, кортежей, записей, полей и т.д.), так и в плане соответствия конкретной модели доступа. Поэтому возникает необходимость разработки собственных решений по управлению доступом. К тому же, если один сервер поддерживает несколько баз данных различной тематики, ведение собственного реестра пользователей и организация разделения доступа является необходимостью.

В этом случае администратором данных самостоятельно формируется реестр объектов предметной области, объединяющий их типы и идентификаторы со структурами (отношениями, атрибутами и записями) базы данных, в которых хранится информация об объектах.

Таблица 1.1. – Команды разграничения доступа в SQL

Команда	Описание	Пример
GRANT	Разрешает роли или пользователю выполнять операции, определенные в момент предоставления разрешения	GRANT SELECT, INSERT, UPDATE ON Sales.Customer TO Sara; GRANT SELECT, UPDATE (Demographics, ModifiedDate) ON Sales.Individual TO Sara; GRANT EXECUTE On dbo.uspGetBillOfMaterials TO Sara;
DENY	Запрещает пользователю или роли определенные разрешения и предотвращает наследование этих разрешений от других ролей	DENY DELETE ON Sales.Customer TO Sara;
REVOKE	Отзывает ранее запрещенные или предоставленные разрешения	REVOKE UPDATE (Demographics) ON Sales.Individual TO Sara;

Безопасность на уровне строк поддерживает два типа предикатов безопасности. Предикаты фильтров автоматически фильтруют строки, доступные для операций чтения (SELECT, UPDATE и DELETE). Предикаты BLOCK явно блокируют операции записи (AFTER INSERT, AFTER UPDATE, BEFORE UPDATE, BEFORE DELETE), которые нарушают предикат.

```
CREATE POLICY doctor_update_policy ON priem
FOR UPDATE TO doctor
USING (priem.doctor_name = current_user);
```

Комбинируя средства разграничения прав доступа в СУБД на уровне атрибутов (GRAND, REVOKE) и на уровне строк (POLICY) можно добиться разграничения на уровне ячеек. В то же время, построение таким образом сложных условий и проверок остается достаточно неудобным и трудно реализуемым. В таких случаях может использоваться разграничение с применением иных средств СУБД – представлений и триггеров, которым посвящена одна из работ далее.

Методические указания по выполнению работы

Задание на работу

Задание на работу включает создание схемы данных и реализация над ней набора прав доступа, как минимум – указанных в варианте выполнения лабораторной работы.

Для выполнения работы рекомендуется СУБД Postgre SQL. Порядок выполнения работы:

1. Реализовать описанные отношения, включая их заданные особенности и связи между ними. Учтите необходимость реализации всех видов ключей и свойств атрибутов. Предложение «если указано» в отношении атрибута показывает, что его отношение в определенные моменты времени работы с базой данных может быть неизвестно.

2. Разработать матрицу доступа к данным детализированную до атрибута на основе предложенных в варианте выполнения работы прав (другие права и типы пользователей можно не рассматривать, на усмотрение обучающегося). Права доступа к данным определить из набора CRUD (Create, Read, Update, Delete). Отобразить ограничения на работу с атрибутами - ограничения горизонтального доступа к данным, если пользователь имеет доступ не ко всем кортежам.

3. Реализовать разграничение доступа при помощи встроенных средств СУБД (права, роли, политики безопасности), для всех ограничений, для которых это возможно. Если реализация какого-либо права (особенности права доступа) встроенными средствами невозможна, дать пользователю более широкие права и отразить это в отчете.

На данном этапе создавать функции, представления или другие вспомогательные структуры для реализации прав доступа не требуется. Если стандартными средствами GRANT и POLICY права доступа согласно составленной матрице задать не получится, считать, что пока их реализация «невозможна».

Содержание отчета

1. Титульный лист.
2. Цель работы.
3. Задание на работу (с указанием данных конкретного варианта).
4. Логическая (реляционная) схема данных в виде рисунка.
5. Код реализации схемы данных.
6. Разработанная матрица доступа со всеми условиями и дополнениями.

7. Матрица доступа в результате применения встроенных средств СУБД (отметить отличия или совпадение с исходной).

8. Код реализации на SQL по крайней мере 2 учетных записей каждого типа, оговоренного в описании прав доступа.

9. Примеры доступа к данным всех приведенных в задании пользователей с соблюдением всех указанных условий.

10. Выводы. Укажите, какие права и/или условия доступа реализовать не удалось, если такие были. Проанализируйте причины.

11. Список литературы.

Варианты выполнения

Каждый вариант выполнения работы включает от двух до трех отношений, в которых указаны текстовые описания атрибутов, первичные (РК), альтернативные (АК) и внешние (FK) ключи. Одинаково обозначенные атрибуты (например, АК1 и АК1) считать относящимися одновременно к одному ключу. По-разному обозначенные (например, АК1 и АК2) – относящимися к разным ключам.

Описание прав доступа и названия ролей представлены в текстовой форме бизнес – логики.

Таблица 1.2. – Варианты выполнения работы

Вар	Описание данных	Описание прав доступа
1.	<i>Отношение 1</i> Номер рецепта(РК), Код заболевания, ФИО пациента(АК1), Дата выписки (АК1), ФИО врача, Срок действия в днях если есть, Наименование лекарства, Код лекарства, Признак того что рецепт использован, Номер аптеки (FK) <i>Отношение 2</i> Номер аптеки (РК), ФИО провизора (АК1), Город аптеки (АК2), Адрес аптеки(АК2)	Врач может видеть все данные рецептов кроме ФИО пациента и кода заболевания. ФИО пациента и код заболевания врач видит только для рецептов, которые выписал сам. Врач может выписывать (добавлять) рецепты и изменять в своих рецептах срок действия. Провизор может видеть все данные рецептов кроме ФИО врача и кода заболевания и может изменять признак использования рецепта и номер аптеки только для еще действующих на момент изменения рецептов.
2.	<i>Отношение 1</i> Номер кабинета (АК1), Код заболевания, ФИО пациента(РК), Дата и время приема (РК, АК1, АК2), ФИО врача (АК2), Дата следующего приема если есть, Описание диагноза	Врач может видеть все данные приемов кроме ФИО пациента, кода заболевания и диагноза. ФИО пациента, код заболевания и диагноз врач видит только для своих прошлых приемов. Врач может только изменять код заболевания и описание диагноза для своих приемов.

Вар	Описание данных	Описание прав доступа
	<i>Отношение 2</i> Код анализа (РК), ФИО пациента (РК, FK1), Дата и время назначения анализа (РК, FK1), Признак готовности анализа, описание результата анализа если есть.	Врач может видеть все данные всех анализов, кроме описания результата, но включая признак готовности, но только для своих пациентов (тех, кто был у него на приеме).
3.	<i>Отношение 1</i> Номер партии лекарства, Код лекарства, Серийный номер пачки лекарства (РК), Число единиц лекарства в пачке, Производитель, Дата окончания срока годности, температура хранения если указана, Признак наркотического содержимого <i>Отношение 2</i> ФИО врача (РК), Дата выписки(АК1), ФИО пациента кому назначено(АК2), Серийный номер пачки лекарства (АК1, АК2, FK, РК), сколько единиц выдано	Врач может видеть все данные по лекарствам, кроме номера партии и серийного номера пачки. По хранимым лекарствам (отношение 1) врач видит только данные по лекарствам с НЕ истекшим сроком годности. Врач видит кто выдал сколько лекарств, но кому – только для своих пациентов. Он может изменять число выданных единиц лекарства и корректировать ФИО пациента (и только их) в существующих записях. А также добавлять новые записи о выдаче, включая ФИО пациента и число выданных единиц лекарства (отношение 2).
4.	<i>Отношение 1</i> Дата и время вызова (АК1, РК), Дата и время приезда к пациенту (АК2, АК3), ФИО пациента (АК2), Адрес пациента (РК), Номер бригады скорой помощи (FK, АК1, АК3), Диагноз пациента, Признак госпитализации пациента, Действия на месте если были выполнены, ФИО дежурного диспетчера <i>Отношение 2</i> Номер бригады скорой помощи (РК), ФИО врача, Должность в бригаде.	Врач может видеть все данные вызовам кроме адресов, ФИО и диагнозов пациентов, выполненных мероприятий. Для своих вызовов врач видит все данные без ограничений. Также для своих вызовов врач может изменять диагноз пациента, признак госпитализации, действия на месте. Дежурный диспетчер видит все данные своих смен и состав бригад кроме диагнозов и действий на месте – но только для вызовов, когда он дежурный.
5.	<i>Отношение 1</i> ФИО сотрудника (РК), Номер документа сотрудника(АК1), Телефон сотрудника (АК2), Дата рождения, Число несовершеннолетних детей, Семейное положение, Уровень образования, Общий стаж, Сведения о квалификации если есть <i>Отношение 2</i> ФИО сотрудника (РК, FK), дата приема на должность (РК), Отдел, Дата окончания работы в должности если есть, Название должности	Сотрудник может видеть все данные о себе, а также ФИО и телефоны других сотрудников, но только для своего отдела где он работает на данный момент. Сотрудник может редактировать только сведения о повышении квалификации, число несовершеннолетних детей и семейное положение для себя самого.

Вар	Описание данных	Описание прав доступа
6.	<i>Отношение 1</i> Название подразделения (РК), Номер отдела (РК, АК1), ФИО руководителя (АК1, FK), Число ставок в отделе, Зарплатный фонд отдела, Число занятых ставок в отделе <i>Отношение 2</i> ФИО сотрудника (РК), Название подразделения(FK1), Номер отдела (FK1), доля занимаемой ставки, название должности, характеристика	Руководитель отдела может видеть все данные о своем отделе и сотрудниках своего отдела, но ничего о сотрудниках других отделов. Если сотрудник работает в нескольких отделах, каждый начальник видит данные по всем отделам. Начальник отдела может корректировать должность и характеристику сотрудника но только для должностей, которые сотрудник занимает в его отделе. Сотрудник видит все данные о себе (но не других сотрудниках) и данные о своем отделе, кроме данных по ставкам и зарплатного фонда.
7.	<i>Отношение 1</i> Код проекта (РК), Название проекта (АК1), Наименование задачи (РК, АК1), ФИО исполнителя (FK), Трудоемкость в часах, Плановая дата выполнения, Реальная дата выполнения если есть, Описание задачи, Отметка о принятии задачи руководителем <i>Отношение 2</i> ФИО сотрудника(РК), Должность, Подразделение, Код проекта которым руководит сотрудник если он есть	Сотрудник видит все названия проектов, задачи без кода проекта и отметки руководителя - для проектов, на которых он работает. Сотрудник может изменять реальную дату выполнения задачи если он является ее исполнителем. Если сотрудник руководит проектом, он видит все данные по этому проекту в отношении всех задач и всех исполнителей и может корректировать отметку о принятии задачи руководителем.
8.	<i>Отношение 1</i> Тип документа (АК), Номер документа (РК), ФИО ответственного (АК), Содержимое документа (JSON), Выпустившее подразделение (АК), Дата ввода в действие (АК), Дата окончания действия если есть, Степень секретности если есть <i>Отношение 2</i> ФИО сотрудника(РК), Номер документа (РК, FK), Дата изменения(РК), Операция с документом	Сотрудник видит все не секретные документы (отношение 1), но не операции с ними (отношение 2). Сотрудник видит все секретные документы, в которых он назначен ответственным. Сотрудник может изменять срок действия документов, в которых он назначен ответственным. Сотрудник видит все операции с документами, которые выполнил он или, если они выполнены в отношении документов, за которые он ответственен. Сотрудник может изменять поле «операция с документом» в отношении 2 для своих записей.

Вар	Описание данных	Описание прав доступа
9.	<i>Отношение 1</i> Инвентарный номер (РК), Наименование (АК1), ФИО материально ответственного (FK1), Помещение где размещено (АК1, FK2), Дата принятия на баланс, Дата снятия с баланса если есть, Срок службы если есть, Балансовая стоимость <i>Отношение 2</i> Код помещения (РК), Код помещения в которое входит данное если оно есть (FK2), ФИО ответственного за помещение (FK1) <i>Отношение 3</i> ФИО сотрудника (РК), Должность	Сотрудник видит все данные о материальных ценностях, кроме даты принятия на баланс, даты снятия с баланса, балансовой стоимости. Для тех ценностей, где сотрудник является материально ответственным лицом, он видит все данные без исключения и может корректировать дату снятия с баланса (и только ее). Ответственный за помещение сотрудник видит все данные о материальных ценностях в этом помещении (в т.ч. из отношения 1), кроме даты снятия с баланса.
10.	<i>Отношение 1</i> Код инцидента безопасности (РК), Подразделение (FK1), Критичность инцидента по классификатору, Дата и время инцидента безопасности (РК, АК1), ФИО заведшего запись об инциденте (АК1, FK2), ФИО ответственного за расследование (FK3), Дата закрытия инцидента если есть, Сумма установленного ущерба если есть, Статус инцидента <i>Отношение 2</i> ФИО сотрудника (РК), Должность, Подразделение (FK) <i>Отношение 3</i> Подразделение (РК), ФИО руководителя (FK)	Сотрудник любой видит Подразделение инцидента, Критичность инцидента по классификатору, Статус инцидента. Сотрудник может завести запись об инциденте. Сотрудник, который завел запись об инциденте, также видит код инцидента, дату и время инцидента. Сотрудник видит все без исключения данные об инцидентах, где он является ответственным лицом или которые произошли в отделе, которым он руководит. Если сотрудник ответственное лицо, он может изменять для инцидента дату закрытия, статус, сумму ущерба.